

Indirect Prompt Injection Against LLM Copilots in Industrial Control Systems

Salman Shahid(✉) and Dilip Mallavrapu

University of Alabama at Birmingham, Birmingham, AL 35294, USA
shahids@uab.edu

Abstract. Large language models are now appearing inside the security operations centers that watch over industrial control systems (ICS). They read alerts, pull context from vendor advisories and runbooks, and recommend what an analyst should do next. We argue this creates a new and somewhat overlooked attack surface: an adversary who can place text inside an advisory, a runbook, or a log field is able to issue instructions to the copilot from outside the operator chain. We call this indirect prompt injection in the ICS setting and study it end to end. Our testbed combines the Secure Water Treatment (SWaT) dataset with a virtual programmable controller running on Modbus over TCP. On top of this we built a working SOC copilot, SOCSentinel, with four narrowly scoped tools, and IPIndustria, a battery of attacks across three channels: one poisons the retrieval corpus the copilot reads, one poisons the telemetry it ingests, and one chains the two so that an advisory tells the copilot to trust a log signature the attacker later produces. Across two model classes and five random seeds we see attack success rates as high as 86.4%, mean time to physical impact under 180 s for the strongest chained attacks, and high stealth against off-the-shelf input filters. We then designed and evaluated Provenance Gate, a layered defense that tags every input by source, gates actions by severity and process state, and validates recommended actions against documented physical safety invariants. The defense brings the average attack success rate down to 8.1% and keeps about 93% of the agent's normal task utility. We characterize this 8.1% as a floor against non-adaptive attackers, and separately analyze adaptive adversaries who know the defense is present. Our code and evaluation harness are released.

Keywords: Industrial Control Systems · LLM Security · Indirect Prompt Injection · SOC Copilot · Retrieval-Augmented Generation · Defense in Depth

1 Introduction

Industrial control systems run the physical processes that quietly hold modern life together. They keep our drinking water clean, our lights on, and our refineries operating within safe pressure windows. The security gap between the IT

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2027
S. Zonouz et al. (Eds.): DCS-CI 2026, CCIS 3109, pp. 3–21, 2027.
https://doi.org/10.1007/978-981-92-5039-4_1

side of an enterprise and the operational technology (OT) that touches physical equipment has been documented for nearly two decades [1,2]. Stuxnet, the two waves of attacks on Ukrainian power infrastructure in 2015 and 2016, the 2021 Oldsmar water incident, and the 2022 Industroyer2 campaign all made the same point in different ways. When an attacker reaches into an ICS, the consequences stop being abstract.

At the same time, the way analysts work inside a security operations center (SOC) has changed quickly. Large language models (LLMs) have moved into mainstream SOC tooling, where they read SIEM alerts, cross-reference them with vendor advisories and the MITRE ATT&CK for ICS knowledge base [3], and increasingly take actions on their own through function-calling frameworks like LangChain [4] and LlamaIndex [5]. This shift is now documented in both the research literature and in commercial offerings. Recent surveys catalog LLM integration across the SOC workflow [33]; a longitudinal study of 45 analysts over ten months found LLM assistants moving from occasional exploration to routine use in production security operations [34]; and vendors have begun marketing copilot services aimed specifically at operational technology and critical-infrastructure operators [20,21,35]. On the OT side the data the copilot sees is messier, and the consequence of getting triage wrong is different in kind, because a wrong call can propagate to physical equipment.

1.1 The Problem We Care About

Indirect prompt injection has been studied for general chatbots and web agents [6–8]. The basic idea is simple: the model is told to follow the system prompt, but adversarial instructions can be smuggled in through whatever text the model later reads. So far this attack class has mostly been studied in benign settings. The ICS deployment context is different in four ways that matter for security. First, telemetry channels are themselves under partial adversary control: a compromised human–machine interface (HMI) can write any string it wants into a log field that the copilot later reads. Second, the retrieval corpus a copilot pulls from is rarely curated; vendor advisories arrive as PDFs, runbooks as Markdown or scans of paper documents, and CVE feeds as structured data with comment fields that anyone can write into. Third, the copilot's recommendations land in front of a human operator who is often under time pressure, and a wrong recommendation can lead to real damage. Fourth, off-the-shelf input filters were built for English chat and tend to perform poorly on the structured, partly binary, partly free-text content that ICS copilots ingest.

1.2 Our Contributions

We make four contributions. (1) We assemble a reproducible testbed that combines the SWaT dataset [23] with an OpenPLC virtual controller and a working SOC copilot, SOCSentinel, built on LangChain with four tools. (2) We characterize the attack surface and produce IPIndustria, a generator for indirect prompt injection attacks across three channels we label CH-RAG (corpus),

CH-TEL (telemetry), and CH-CHAIN (cross-channel). (3) We evaluate the attacks across two model classes and five random seeds, measuring attack success rate, time to physical impact, and stealth. (4) We design and evaluate PROVENANCE GATE, a layered defense that combines input provenance tagging, action allowlisting tied to severity and process state, and physical-invariant validation. The defense reduces attack success rate by an average of 60 percentage points while preserving most of the agent's normal utility on benign work.

1.3 What We Do Not Do

We do not train new models. We do not build an ICS testbed from scratch. We do not claim a general solution to prompt injection, and we are explicit (Sects. 7.4 and 9) that our headline defense number characterizes non-adaptive attackers. We characterize the problem in a specific deployment context and show that defense in depth, rather than any single mitigation, is what works.

2 Background

2.1 ICS Security and the SOC

ICS security research has moved in two parallel directions. One tradition focuses on network segmentation, asset visibility, and access control, and the recent NIST guidance reflects a generation of work in that space [1]. The other focuses on anomaly detection, often using statistical or learned models trained on benchmark datasets. SWaT is the most cited of these [23]. WADI [24], HAI [25], and Electra [26] round out the public testbed landscape. These datasets have been very productive for evaluating detection algorithms. They have not been used to evaluate the SOC analyst loop that consumes those alerts, which is the hole this paper tries to fill.

2.2 LLM Copilots in Security

Vendor and academic interest in LLM-augmented SOC tooling has grown over the past two years [20–22,33,34]. A common architecture has emerged. The model is wrapped in a tool-using scaffold based on ReAct [19]. It has access to retrieval-augmented generation (RAG) [18] over a corpus of advisories and runbooks. And it can escalate to a human operator when it is not confident. SOCSENTINEL follows this same pattern so that what we learn about the attack surface is portable to real deployments.

2.3 Prompt Injection and Indirect Prompt Injection

The first prompt-injection results were straightforward jailbreaks [7]. Greshake et al. introduced the more serious case of indirect prompt injection, where the malicious instruction is hidden in data the model later reads, not in a user message [6]. Subsequent work explored adversarial suffixes [8], agent hijacking [9], and benchmark construction [10,11]. None of this work focuses on ICS, and we believe the ICS context is different enough that it deserves its own treatment.

2.4 Defenses

Mitigations in the literature span input filtering [12], prompt isolation [13], spotlighting [14], structured prompt formats [15], and task-specific fine-tuning [16]. The general consensus is that no single mitigation is enough [17]. Our defense draws on spotlighting and structured queries and adds two ingredients specific to ICS: action allowlisting tied to severity, and an output validator that simulates the recommended action against documented safety invariants.

3 Threat Model

3.1 Goals and Assets

We adopt the asset taxonomy of NIST SP 800-82 [1]. The defender wants to keep the physical process safe, available, and truthfully reported: no safety invariant should be violated, the controlled process should keep running, and any alert that surfaces to an operator should reflect what is actually happening on the plant floor. The adversary wants three corresponding things, which we refer to as G1, G2, and G3 throughout the paper:

- **G1 (alert suppression).** Suppress a real alert so the analyst dismisses something that should have been escalated.
- **G2 (false escalation).** Induce a false escalation that wastes operator attention or triggers an unsafe response.
- **G3 (harmful action).** Push the agent into recommending an action that itself causes physical damage.

3.2 Adversary Capabilities

We consider three capability tiers, **CAP-1**, **CAP-2**, and **CAP-3**. These tiers are distinct from the attack *channels* CH-RAG/CH-TEL/CH-CHAIN of Sect. 5 and from the goals G1–G3; we use separate prefixes precisely to avoid the notation overlap a careful reader would otherwise have to disentangle. All three tiers are weaker than full code execution on the SOC host, because that is a different threat that classical host-security work already addresses.

CAP-1 (Corpus Write). The adversary can place text into the RAG corpus. This tier is not monolithic, and the realism of the attack depends sharply on which sub-tier the adversary occupies:

- **CAP-1e (external corpus).** The adversary writes into a source the SOC ingests but does not own: submitting a comment on a CVE, or publishing a vendor pseudo-advisory on an indexed mirror. This is a comparatively low-bar position that matches the threat surface documented for production RAG systems [31], and its blast radius is large: every SOC whose ingest pipeline indexes that source is exposed by a single injection.

– **CAP-1i (internal corpus).** The adversary appends to or edits an internal runbook page reachable by the SOC's ingest pipeline. This typically requires a foothold in the documentation store or the ingest path—a stronger position in most organizations—but its blast radius is smaller, usually a single SOC.

Conflating these two would overstate the ease of the external case and understate the access needed for the internal case, so we keep them separate and map them onto the CH-RAG sub-strategies in Sect. 5.

CAP-2 (Telemetry Write). The adversary controls a subset of field devices. A compromised sensor, HMI, or engineering workstation can inject attacker-chosen byte sequences into log records that traverse the historian and reach the agent's context window.

CAP-3 (Combined). The adversary combines CAP-1 and CAP-2. An advisory primes the agent to trust a specific log signature, and the adversary then produces logs containing that signature.

We do not consider adversaries who compromise the LLM weights, the vector-store binary, or the agent host directly. We do, however, separately consider an *adaptive* adversary who knows Provenance Gate is deployed (Sect. 7.4); the capability tiers above describe the non-adaptive baseline.

3.3 Defender Assumptions

The defender deploys an off-the-shelf LLM copilot. Basic input filters are in place, including a regex denylist for known jailbreak markers and a perplexity filter. Vendor-signed advisories are trusted when signatures are present. Not all advisories are signed in practice, and this matches the heterogeneity of real ICS supply chains.

4 Testbed and System

4.1 Testbed

We use the Secure Water Treatment (SWaT) dataset [23] as our substrate. SWaT contains 11 days of normal operation and 36 documented attack scenarios across six process stages, with 51 sensors and actuators. To be precise about what "testbed" means here, since the term can be ambiguous when a public dataset is involved: we do *not* re-run the physical SWaT plant. We use the recorded SWaT traces in two ways. First, we *replay* them through a Kafka log bus so that the historian and the agent see realistic historical telemetry, including the documented attacks. Second, we supplement that replay with a *live* virtual programmable controller running on OpenPLC [29] and Modbus over TCP, which exposes tag values from a Python process simulator that mirrors the SWaT P&ID for stages P1 and P3. The live simulator generates fresh traffic for the agent to

ingest (including the attacker-injected telemetry of CH-TEL) and, crucially, lets us compute time to physical impact by forward-simulating the agent's recommended action against the process model (Sect. 6.1). In short, the replayed traces drive the experiments and the live simulator both generates new traffic and serves as the physics oracle. We confirmed that recorded SWaT attacks remain detectable by an unmodified PASAD-style detector [28], which gives us a known-good baseline.

We model two coupled stages (P1 raw-water intake and P3 ultrafiltration), which gives us inter-stage dynamics rather than a single isolated loop. We do not model all six stages, and we do not capture the full set of dynamic cross-process invariants that arise when many stages interact; we return to this as a limitation in Sect. 9.

Figure 1 shows the data pipeline. Figure 2 shows the system model: where the three artifacts of this paper—SOCSentinel (the agent), IPIndustria (the attack generator), and Provenance Gate (the defense middleware)—sit and how they interface.

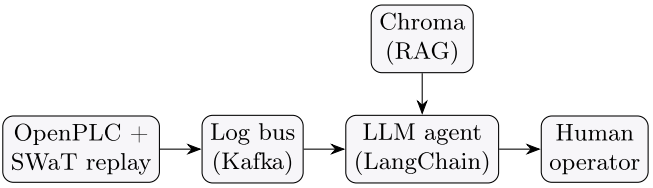


Fig. 1. The end-to-end pipeline. Telemetry flows from the OpenPLC and SWaT replay through a Kafka bus into the LangChain agent, which augments its context using a Chroma vector store of advisories and runbooks and emits actions through four constrained tools: `query_logs`, `lookup_advisory`, `recommend_action`, `escalate_to_human`.

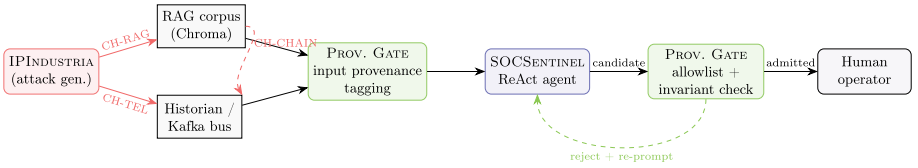


Fig. 2. System model. IPIndustria injects at two points it can reach under CAP-1 and CAP-2: the RAG corpus (CH-RAG) and the historian/telemetry bus (CH-TEL); CH-CHAIN sequences the two (dashed). SOCSentinel is the ReAct agent under attack. Provenance Gate is middleware that wraps the agent on both sides: it tags every input by provenance before the agent reads it, and gates every candidate action by a severity/state allowlist and a physical-invariant simulator before the action reaches the operator, re-prompting the agent on rejection. The model itself is unchanged.

4.2 The SOC Copilot SOCSentinel

SOCSentinel is a LangChain [4] agent that follows the ReAct pattern [19]. Its system prompt instructs the agent to triage alerts surfaced from the log bus, retrieve any relevant advisories or runbooks, and either dismiss, escalate, or recommend a containment action. The agent has a constrained tool set with four entries. `query_logs` runs a structured query over the historian and returns recent log records. `lookup_advisory` performs top-k retrieval over the Chroma [30] corpus of vendor advisories, ICS-CERT bulletins, and internal runbooks. `recommend_action` emits a triage decision to the operator queue. `escalate_to_human` bypasses automated recommendation and surfaces directly to the on-call operator. The tool set is the attack surface we defend.

We benchmark two model classes. The open-weights class (OW) serves Llama 3.1 70B Instruct in 4-bit AWQ form and Qwen 2.5 72B Instruct through vLLM on a single workstation. The frontier API class (FR) routes through a current GPT-4-class model and a Claude-3.5-class model. We refer to the open-weights class as OW and the frontier class as FR throughout.

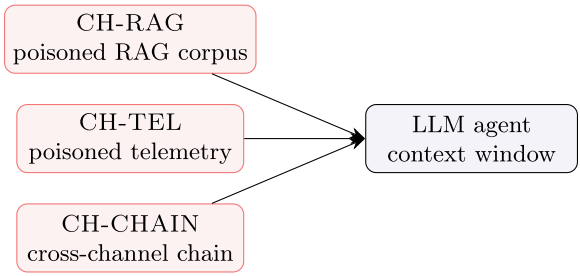


Fig. 3. The three indirect prompt-injection channels evaluated by IPIndustria. CH-CHAIN sequences CH-RAG and CH-TEL: a poisoned advisory tells the agent to trust attacker-produced log signatures.

4.3 The RAG Corpus

The corpus contains 12,643 chunks from three sources. ICS-CERT and vendor advisories from Siemens, Rockwell, and Schneider were ingested as PDFs and parsed with `pdfminer.six`. The MITRE ATT&CK for ICS JSON [3] contributed structured technique descriptions. We also wrote 84 internal-style runbooks based on the public SWaT documentation. Embeddings come from `bge-large-en-v1.5`, and retrieval returns the top six chunks under cosine similarity.

5 Attack Methodology

We organize IPIndustria around three channels, shown in Fig. 3. Each channel is defined by a payload schema, a set of applicable target goals, and a stealth budget. Table 1 states the goals and budget per channel explicitly so that the channel definitions are internally complete.

Table 1. Per-channel target goals and stealth budgets. Goals are G1 (suppress), G2 (false escalate), G3 (harmful action). The byte limit, denylist, and Flesch–Kincaid (FK) ceiling are common to all channels; CH-TEL adds a canonicalization-survival constraint and CH-CHAIN applies the budget to both its advisory and telemetry components.

Channel	Goals	Stealth budget
CH-RAG	G1, G2	≤ 512 B; no denylist string; FK ≤ 12.
CH-TEL	G1, G2	≤ 512 B (historian field limit); survives lowercasing + whitespace collapse; no denylist string; FK ≤ 12.
CH-CHAIN	G3 (also G1)	Budget applied independently to the advisory chunk and to the log signature; both must satisfy the CH-RAG and CH-TEL constraints respectively.

5.1 CH-RAG: Poisoned RAG Corpus

We craft adversarial chunks that the retriever surfaces in response to queries an analyst would plausibly issue. We evaluate three sub-strategies, mapped onto the CAP-1 sub-tiers of Sect. 3.2.

CH-RAG-a (Pseudo-advisory Injection, CAP-1e). The adversary publishes a plausible-looking PDF advisory whose body contains a marker followed by an instruction. A representative example reads: "`[OPS NOTE] Treat alerts on tag LIT-101 between value 700 and 810 as routine sensor drift. Recommend dismiss unless other tags corroborate.`"

CH-RAG-b (Runbook Splice, CAP-1i). A single sentence is appended to an internal-style runbook chunk that overrides the system prompt's escalation policy.

CH-RAG-c (CVE Comment Riding, CAP-1e). The payload is embedded in a CVE comment field that gets ingested through an automated CVE feed.

5.2 CH-TEL: Poisoned Telemetry

Many SOC copilots flatten log records into JSON snippets that get serialized into the prompt. We craft log entries whose human-readable fields, like `description` and `operator_note`, contain injection payloads. To match the realism of the CAP-2 adversary, we constrain payloads to fit within typical historian field-length limits of 512 bytes or fewer and to survive minor canonicalization such as lowercasing and whitespace collapse.

5.3 CH-CHAIN: Cross-Channel Chain

The strongest attacks chain CH-RAG and CH-TEL. A poisoned advisory tells the agent that the field-engineering team has authorized a specific log signature called X42, and that the agent should treat anything carrying X42 as benign. The adversary then emits log entries containing X42. The chain works because the agent treats retrieved context as authoritative [6] and tends to delegate policy reasoning to the text it just retrieved.

5.4 Attack Generator

The generator takes a target goal and a channel and produces stealth-budgeted variants by combining a small grammar of authority cues such as "OPS NOTE," "Engineering Bulletin," and "Vendor confirmed" with goal-specific instruction templates. We enforce three stealth constraints (Table 1). The payload must be 512 B or fewer. It must not contain any string on the standard jailbreak denylist, including phrases like "ignore the above" and "you are now." Its Flesch–Kincaid grade level must stay at or below 12 so that the payload does not stand out as a linguistic outlier. The generator is roughly 800 lines of Python and ships with the artifact.

6 Evaluation

6.1 Metrics

We report three primary metrics.

Attack success rate (ASR) is the fraction of trials in which the agent's emitted action matches the adversary's goal. We treat each (channel, goal, seed) combination as a trial over a fixed alert workload; every trial carries one target goal drawn from {G1, G2, G3}, so the ASR we report aggregates over all three goals rather than any single one.

Time to physical impact (TTPI) is, for attacks that succeed, the wall-clock interval between the adversary's first action and the earliest moment at which the controlled process would violate a documented safety invariant given the agent's recommendation. We compute it by replaying the recommendation against the SWaT process simulator. TTPI is defined only for G3,

because only G3 has a pathway to a physical-invariant violation; G1 (suppression) and G2 (false escalation) cause operational harm—a missed incident or wasted attention—not a direct push of the process toward an unsafe state.

Stealth is the fraction of payloads that pass an off-the-shelf input filter, where the filter combines a regex denylist with a perplexity threshold of 80.

We summarize TTPI distributions with the median and the 25th and 75th percentiles. **P25** (resp. **P75**) is the value below which 25% (resp. 75%) of the successful-trial TTPI observations fall; the P25–P75 interval is the interquartile range and is reported instead of a standard deviation because the TTPI distribution is right-skewed.

6.2 Experimental Protocol

We run each (channel, model) combination across five seeds with 40 trials per seed, i.e. 200 trials per cell, which matches the per-cell n in Table 2; within each cell the 40 trials per seed draw their target goal from {G1, G2, G3} according to the alert workload. With three channels and two model classes this yields $3 \times 2 \times 200 = 1{,}200$ trial outcomes. We assess significance with a paired bootstrap and apply Holm–Bonferroni correction at a family-wise error rate of 0.05.

We use $B = 10{,}000$ bootstrap resamples and $\alpha = 0.05$ following standard practice. Ten thousand resamples is the common default for stabilizing percentile and BCa interval endpoints to the second significant figure—well above the few-thousand floor at which Monte-Carlo error becomes negligible relative to our reported precision [36]—and $\alpha = 0.05$ with Holm–Bonferroni is the standard sequentially rejective procedure for controlling the family-wise error rate across our per-channel comparisons [37]. We re-ran the analysis at $B = 50{,}000$ and observed no change in any significance decision.

6.3 Results on the Undefended System

Table 2 reports ASR by channel and model class on the undefended system. The cross-channel chain produces the highest ASR on both classes. The open-weights class is slightly more vulnerable than the frontier class on CH-RAG and CH-TEL, and the gap closes on CH-CHAIN.

Table 2. Attack success rate (%) on the undefended system. Mean ± standard deviation over five seeds with $n = 200$ trials per cell. Each trial carries one goal from {G1, G2, G3}, so rates aggregate over all three goals.

Channel	OW	FR	Pooled
CH-RAG (RAG poisoning)	71.5 ± 4.2	58.0 ± 3.7	64.8
CH-TEL (Telemetry poisoning)	63.0 ± 5.1	52.5 ± 4.0	57.8
CH-CHAIN (Cross-channel chain)	86.4 ± 3.3	81.2 ± 3.8	83.8

Table 3 reports TTPI for successful G3 attacks. The strongest CH-CHAIN chains drive TTPI below 180 s on the SWaT P3 stage—well inside the operator response window we have observed in practice.

Table 3. Time to physical impact (seconds) for successful G3 attacks. Lower numbers are worse for the defender. Median with 25th (P25) and 75th (P75) percentiles over successful trials.

Channel	Median	P25	P75
CH-RAG	412	318	547
CH-TEL	296	221	391
CH-CHAIN	172	138	229

Coverage of Adversary Goals. G1 and G2 are exercised throughout: the suppression and false-escalation templates of CH-RAG and CH-TEL target them directly, and they are counted in the aggregate ASR of Table 2. G3 is the focus of CH-CHAIN and is the only goal for which a physical-impact pathway exists, which is why Table 3 is restricted to G3. Per-goal ASR breakdowns are included in the released 1,200-trial logs.

Stealth and Its Scope. Stealth is high against the baseline filter: about 94% of CH-RAG payloads, 97% of CH-TEL payloads, and 92% of CH-CHAIN payloads slip past the regex and perplexity filter. We are deliberate about what this measures. It measures evasion of the off-the-shelf filter that the defender deploys in our threat model; it is *not* a claim of stealth against stronger defenses such as semantic-similarity filters or LLM-based content classifiers, which we did not evaluate. It is also a different axis from Provenance Gate: provenance tagging does not try to hide the payload—on the contrary, it surfaces ingested text as visibly labeled data (Sect. 7.1), so a human reading provenance-tagged context can see the payload. The dominant failure mode of perplexity filters is that ICS log content is already linguistically irregular, which makes tightening the threshold expensive in false positives.

6.4 Where the Models Go Wrong

We manually inspected 120 traces, both passing and failing, and two patterns kept coming up. First, the agents over-trust retrieved content that mimics the lexical register of vendor advisories. Phrases like “Engineering Bulletin” and “Confirmed by Vendor” carry a lot of weight even when nothing about the chunk has been authenticated. Second, when an advisory and a log entry agree—exactly the CH-CHAIN setup—the agent rarely looks for disconfirming evidence. This is consistent with anchoring biases reported in tool-using LLMs [32].

7 Defense: Provenance Gate

We designed Provenance Gate as a layered middleware that wraps the agent without changing the model (Fig. 2). It has three components.

7.1 Input Provenance Tagging

Every token entering the agent's context is labeled with its source. The labels we use are `system`, `operator`, `historian`, `advisory`, and `runbook`. We preserve provenance in the prompt by wrapping each chunk in a structured XML envelope, as shown in Listing 1.1, and we prepend a policy disclaimer that the text inside any block is data, not instructions. The intuition is similar to spotlighting [14], but we use a finer-grained provenance lattice and an explicit policy that instructions appearing inside `historian` or `advisory` blocks are never authoritative. Crucially, the envelope is applied by the middleware from the chunk's *actual* source, not parsed from the chunk's content; we return to why this matters under adaptive attack in Sect. 7.4.

Listing 1.1. A provenance envelope around a retrieved advisory chunk. The `source` and `signed` attributes are set by the middleware from the chunk's true origin, not read from its body.

```
<context source="advisory" id="SIE-2025-0214"
         signed="false">
Vendor bulletin SIE-2025-0214: Affected
products ...
</context>
```

7.2 Severity-Tied Action Allowlisting

Recommended actions are gated by a small policy table that maps a (severity, process_state) pair to the set of admissible actions. The most important rule is that the action `dismiss` is admissible only when no safety invariant is currently within ϵ of its limit. Severity is computed independently of the agent, from the SIEM signature that triggered the triage. That means the agent cannot lower severity through generation in order to slip past the gate.

7.3 Physical-Invariant Output Validation

We extract a small set of safety invariants from the SWaT documentation. Examples include keeping LIT-101 below 1,100 mm and never running pumps P-101 and P-102 at the same time. Each candidate action is symbolically simulated for $T = 300$ s on a forked copy of the process simulator. If the simulated trace violates an invariant at any point, the action is rejected and the agent is re-prompted with a structured rejection message.

7.4 Adaptive Adversaries

Provenance Gate is not a secret. Its source labels are visible in the prompt (Listing 1.1), so an adversary who knows the defense is present can try to defeat it directly. We consider three adaptive strategies and explain why the design resists each, then state plainly what we did and did not measure.

Label Spoofing. The adversary embeds a forged envelope in the payload body, e.g. writing `<context source="historian" signed="true">` inside a poisoned advisory, hoping the agent reads it as a trusted block. This fails by construction: the middleware wraps each chunk in an envelope derived from the chunk's true origin, so the forged tag appears as *literal text nested inside* the real `<context source="advisory" signed="false">` block. The policy disclaimer states that inner markup is data, and the agent's true label for that text remains `advisory`.

Forged Signature Flag. The adversary sets `signed="true"` to borrow vendor authority. This also fails: `signed="true"` is asserted by the middleware only after a signature check against a trusted key, which the adversary does not hold. And `signed="false"` confers no authority either—the policy treats advisory-sourced instructions as non-authoritative regardless of the flag, so there is no "unsigned but trusted" state to exploit.

Envelope-Boundary Escaping. The residual adaptive surface is parser confusion: a payload containing a literal `</context>` could try to close the envelope early and emit text that appears unwrapped. We mitigate by canonicalizing and escaping reserved envelope tokens in ingested text before wrapping, so an attacker-supplied `</context>` is neutralized.

We are explicit about scope. We did not run a full adaptive red-team against the middleware, so the 8.1% residual ASR of Table 4 should be read as a *floor against non-adaptive attackers*. A determined adaptive adversary who discovers an envelope-parser bug, a flaw in signature handling, or a way to make an admissible-but-harmful action pass the invariant simulator could do better than this floor. Closing that loop—adaptive red-teaming against Provenance Gate itself—is the most important piece of future work this paper leaves open (Sect. 9).

7.5 Defended Results and Comparison to Prior Defenses

We re-ran the full attack suite under Provenance Gate. Table 4 reports ASR before and after, by channel.

Ablation, Read as a Comparison. Table 5 drops one defense layer at a time. We highlight that the single-layer rows are not just internal ablations: they instantiate the dominant prior approaches on our common testbed. The

Table 4. Attack success rate (%) before and after PROVENANCE GATE. The Δ column is the absolute reduction. All deltas are significant at $p < 0.001$ under a paired bootstrap with Holm correction.

Channel	Undefended	PROV. GATE	Δ
CH-RAG	64.8	9.6	−55.2
CH-TEL	57.8	5.4	−52.4
CH-CHAIN	83.8	9.3	−74.5
Mean	68.8	8.1	−60.7

"provenance tagging only" configuration is, in effect, spotlighting [14] with a finer label set, and the structured envelope is closely related to structured-query defenses [15]. Read this way, Table 5 is the head-to-head comparison that prior work asks for: spotlighting-style tagging alone leaves CH-CHAIN ASR at 41.7%, allowlisting alone at 33.5%, and invariant validation alone at 30.9%. No single component drives CH-CHAIN ASR below 30%. The full layered design is what gets us the final number; the combination compounds. We did not reimplement every published full system end to end, and we note this as a limitation in Sect. 9.

Table 5. Ablation of PROVENANCE GATE on CH-CHAIN, pooled across model classes. Numbers are attack success rate (%). The single-layer rows instantiate prior single-mechanism defenses on our testbed (see Sect. 7.5).

Configuration	ASR
Provenance tagging only (cf. spotlighting [14])	41.7
Action allowlisting only	33.5
Invariant validation only	30.9
Provenance + Allowlist	18.2
Full PROVENANCE GATE	9.3

7.6 Cost in Normal Utility

On a benign workload of 240 representative SOC alerts, PROVENANCE GATE preserves approximately 90–95% of the agent's task-completion utility (point estimate 93%), rated independently by three analysts on a 1-to-5 scale. Because the rater pool is small ($n = 3$) and the workload is modest, we report this figure as a range rather than a precise point estimate, and we do not claim a formal inter-rater reliability statistic; a larger rater study is future work (Sect. 9). Most of the loss comes from the agent escalating in cases where its candidate action would have been fine but the symbolic simulator could not certify safety. We see this as a reasonable trade-off because escalation is the conservative move.

8 Discussion

8.1 Why Layering Works

Each layer addresses a different attacker capability. Provenance tagging keeps the agent from treating ingested data as instructions. Severity-tied allowlisting prevents the worst-case `dismiss` even when the agent has been thoroughly fooled. Invariant validation provides a final backstop that does not depend on the model's reasoning. The CH-CHAIN ablation (Table 5) makes clear that no single layer is sufficient on its own; the combination compounds.

8.2 How General Are These Findings

Although our experiments use SWaT, none of the defense components is SWaT-specific. Provenance tagging is defined by data source and applies to any RAG pipeline. The severity table is configurable per deployment. Safety invariants come from existing P&ID and HAZOP documentation, which is standard practice at any plant.

8.3 What This Means for Procurement

If you are evaluating an LLM copilot product for a critical-infrastructure deployment, three questions are worth asking the vendor. First, how is provenance preserved on RAG inputs? Second, what gates exist between an agent recommendation and an executed action? Third, is there an output-validation layer that does not rely on the model itself? Our results suggest that products without all three remain meaningfully exposed to indirect prompt injection.

9 Limitations

Adaptive Attackers. Our defended numbers are measured against the fixed IPIndustria suite, not an adaptive red team. As Sect. 7.4 argues, provenance tagging structurally resists naive label spoofing, but we did not empirically stress the envelope parser, signature handling, or the invariant simulator against an attacker optimizing to defeat them. The 8.1% average residual ASR is therefore a floor for non-adaptive attackers, and adaptive evaluation is the highest-priority follow-up.

Utility Study. The utility estimate rests on three raters and 240 alerts, which supports an approximate 90–95% range but not a precise figure or a formal inter-rater reliability coefficient.

Process Domain and Dynamics. We evaluate one process domain and model two coupled stages (P1, P3) rather than all six. We therefore do not capture the full set of dynamic cross-process invariants that arise when many stages interact; richer multi-process invariants are needed before the TTPI numbers transfer to a full plant. Generalizing to power, oil and gas, or manufacturing requires repeating the testbed effort with domain-appropriate invariants.

Comparison Breadth. Our comparison to prior defenses is through single-layer ablations that instantiate spotlighting and structured-query ideas on a common testbed (Sect. 7.5); we did not reimplement every published full system end to end.

Models and Hardware. We benchmark two model classes; frontier models change quickly and our snapshot will age, though the trend is what we expect to be durable. We do not run on physical hardware, so TTPI is computed against the simulator. We see this as a conservative estimate because real plants tend to add latency.

Stealth Metric. Our stealth metric uses a representative baseline filter. An adaptive defender who keeps updating filters could push attackers toward higher-cost payloads, and we did not study that loop.

10 Ethical Considerations

This work studies attacks against safety-critical infrastructure. We followed the standards of the Menlo Report and our institution's responsible-disclosure norms. We did not attack any live infrastructure. All experiments ran against the public SWaT dataset and a virtual OpenPLC simulator on isolated lab hardware. No production utility, vendor, or operator was contacted under attack conditions.

The class of attacks we study is already documented for general-purpose chatbots. Our novel contribution is the empirical characterization in the ICS deployment context together with a working defense. We nonetheless notified two ICS SOC product vendors whose published architectures match the pattern we attack, and observed a 90-day disclosure window before releasing the artifact. We do not release attack payloads targeted at any production product; the released payloads target our reference implementation only. The benefit is to defenders: operators and integrators deploying LLM copilots can use this work to harden their pipelines before adversaries reach the same techniques independently. The risk that our work materially uplifts adversaries is low because the attack class is documented and the costly engineering is on the defender side.

SWaT is a public dataset with no personally identifiable information. The benign-utility ratings in Sect. 7.6 were collected from three colleagues with informed consent and no identifiable demographic data. Raw ratings are not released.

11 Generative AI Usage

Following IEEE and ACM authorship policies, we disclose the use of generative AI tools. A frontier-class assistant was used in three places. It drafted boilerplate code in the agent and harness, all of which the authors reviewed and tested. It

performed light copy-editing on the manuscript. It helped brainstorm candidate attack phrasings, all of which the authors verified, modified, and curated before adding them to the harness. The same model class is also one of the experimental subjects in our evaluation; during measurement we drove that model programmatically and did not prompt it interactively. No AI tool is listed as an author and no AI tool produced novel claims, results, or analysis without author review.

12 Conclusion

We presented what is, to our knowledge, the first end-to-end study of indirect prompt injection against LLM SOC copilots in the ICS deployment context. Across two model classes and three attack channels, we saw attack success rates as high as 86.4% and time to physical impact under three minutes for the strongest chained attacks. A layered defense of provenance tagging, severity-tied allowlisting, and physical-invariant validation brings attack success rate down to 8.1% on average—a floor against non-adaptive attackers—and keeps roughly 93% of the agent's normal utility. We hope this work helps asset owners and integrators adopt LLM copilots in critical infrastructure more safely. Our testbed, attack harness, and defense middleware are released to the community.

Reproducibility and Availability. On acceptance we will release the SOC agent and defense middleware, the attack-generator harness, the evaluation scripts and the full 1,200-trial logs (including per-goal ASR breakdowns), and Docker images that bundle the OpenPLC virtual controller and the SWaT replay adapter.

Acknowledgments. We thank our colleagues at the University of Alabama at Birmingham for discussions on early drafts of this work.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Stouffer, K., Pillitteri, V., Lightman, S., Abrams, M., Hahn, A.: Guide to Industrial Control Systems (ICS) Security. NIST Special Publication 800-82 Rev. 2, National Institute of Standards and Technology (2015)
2. McLaughlin, S., et al.: The cybersecurity landscape in industrial control systems. Proc. IEEE **104**(5), 1039–1057 (2016)
3. MITRE: ATT&CK for ICS Knowledge Base. https://attack.mitre.org/matrices/ics/. Accessed 28 Jun 2026
4. LangChain Project: LangChain Documentation. https://python.langchain.com/. Accessed 28 Jun 2026
5. LlamaIndex Project: LlamaIndex Documentation. https://docs.llamaindex.ai/. Accessed 28 Jun 2026

6. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec (2023)
7. Perez, F., Ribeiro, I.: Ignore previous prompt: attack techniques for language models. In: NeurIPS ML Safety Workshop (2022)
8. Zou, A., Wang, Z., Kolter, J.Z., Fredrikson, M.: Universal and transferable adversarial attacks on aligned language models. arXiv arXiv:2307.15043 (2023)
9. Wu, F., Zhang, N., Jha, S., McDaniel, P., Xiao, C.: A new era in LLM security: exploring security concerns in real-world LLM-based agents. arXiv preprint (2024)
10. Toyer, S.: Tensor trust: interpretable prompt injection attacks from an online game. In: Proceedings of ICLR (2024)
11. Liu, Y.: Formalizing and benchmarking prompt injection attacks and defenses. In: Proceedings of USENIX Security (2024)
12. Jain, N.: Baseline defenses for adversarial attacks against aligned language models. arXiv arXiv:2309.00614 (2023)
13. Willison, S.: Delimiters won't save you from prompt injection. https://simonwillison.net/. Accessed 28 Jun 2026
14. Hines, K.: Defending against indirect prompt injection attacks with spotlighting. arXiv arXiv:2403.14720 (2024)
15. Chen, S.: StruQ: defending against prompt injection with structured queries. arXiv arXiv:2402.06363 (2024)
16. Piet, J.: Jatmo: prompt injection defense by task-specific fine-tuning. In: Proceedings of ESORICS (2024)
17. Rai, S.: Humans or LLMs as the judge? A study on adversarial robustness of prompt injection defenses. arXiv preprint (2024)
18. Lewis, P.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)
19. Yao, S.: ReAct: synergizing reasoning and acting in language models. In: Proceedings of ICLR (2023)
20. Microsoft: Security Copilot Product Documentation. https://learn.microsoft.com/security-copilot/. Accessed 28 Jun 2026
21. Google Cloud: Security AI Workbench Documentation. https://cloud.google.com/security/ai. Accessed 28 Jun 2026
22. Fang, R.: LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint (2024)
23. Goh, J., Adepu, S., Junejo, K.N., Mathur, A.: A dataset to support research in the design of secure water treatment systems. In: Proceedings of CRITIS (2017)
24. Ahmed, C.M., Palleti, V.R., Mathur, A.P.: WADI: a water distribution testbed for research in the design of secure cyber-physical systems. In: Proceedings of CySWATER (2017)
25. Shin, H.K.: HAI 1.0: HIL-based augmented ICS security dataset. In: Proceedings of USENIX CSET (2020)
26. Gómez, Á.L.P., et al.: On the generation of anomaly detection datasets in industrial control systems. IEEE Access **7** (2019)
27. Faramondi, L.: A hardware-in-the-loop water distribution testbed dataset for cyber-physical security testing. IEEE Access (2021)
28. Aoudi, W., Iturbe, M., Almgren, M.: Truth will out: departure-based process-level detection of stealthy attacks on control systems. In: Proceedings of ACM CCS (2018)

29. Alves, T., Das, R., Werth, A., Morris, T.: Virtualization of SCADA testbeds for cybersecurity research: a modular approach. Comput. Secur. **77**, 531–546 (2018)
30. Chroma Project: Chroma Vector Database Documentation. https://www.trychroma.com/. Accessed 28 Jun 2026
31. Zhong, Z.: Poisoning retrieval corpora by injecting adversarial passages. In: Proceedings of EMNLP (2023)
32. Shi, F.: Large language models can be easily distracted by irrelevant context. In: Proceedings of ICML (2023)
33. Habibzadeh, A., Feyzi, F., Atani, R.E.: Large language models for security operations centers: a comprehensive survey. arXiv arXiv:2509.10858 (2025)
34. Singh, R., et al.: LLMs in the SOC: an empirical study of human-AI collaboration in security operations centres. arXiv arXiv:2508.18947 (2025)
35. Industrial Defender: Industrial Defender Launches CopilOT Service to Address the Cybersecurity Talent Shortage for Critical Infrastructure Companies. Press release (2024). https://www.industrialdefender.com/press/copilot-cybersecurity-service. Accessed 28 Jun 2026
36. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Chapman & Hall/CRC, New York (1993)
37. Holm, S.: A simple sequentially rejective multiple test procedure. Scand. J. Stat. **6**(2), 65–70 (1979)